MCP Gateway: Downstream API Tokens

Oct 29, 2026 · @Mark Gerow

Implementation plan for letting adapter MCP servers and registered tool backends call downstream APIs as the authenticated caller, via on-behalf-of token exchange.

Goal and scope

An adapter MCP server or a registered tool backend must be able to call a downstream API as the authenticated caller. The same adapter image must also work standalone, served directly to a client app, so whatever carries identity and tokens has to work in both deployments. Neither can today: the gateway strips the inbound `Authorization` header and nothing in the solution performs a token exchange.

In scope:

- An on-behalf-of (OBO) exchange so each backend receives a token scoped to its own declared downstream API, never one with the gateway's audience.
- A `DownstreamScopes` declaration per adapter and per tool, gated at registration and immutable after create.
- A cached token service keyed on caller, resource and scope.
- A confidential-client credential for the gateway, which has none today.
- A signed identity token, so the same adapter image authenticates its caller whether it runs behind the gateway or standalone.

Out of scope:

- The existing per-resource `requiredRoles` authorization, which keeps working unchanged and stays the primary boundary.
- `useWorkloadIdentity`, which grants a pod its own managed identity and has nothing to do with the caller.
- Identity providers other than Entra ID. The gateway is Entra-only today and this plan does not generalise it.
- Direct access to tool backends. Tools are callable only through the gateway, by decision, so their backends stay cluster-internal and gain no caller-facing authentication.

- The shape of a tool backend's request body. It keeps receiving the arguments object as JSON; this plan adds one header, it does not change the contract.

Current state

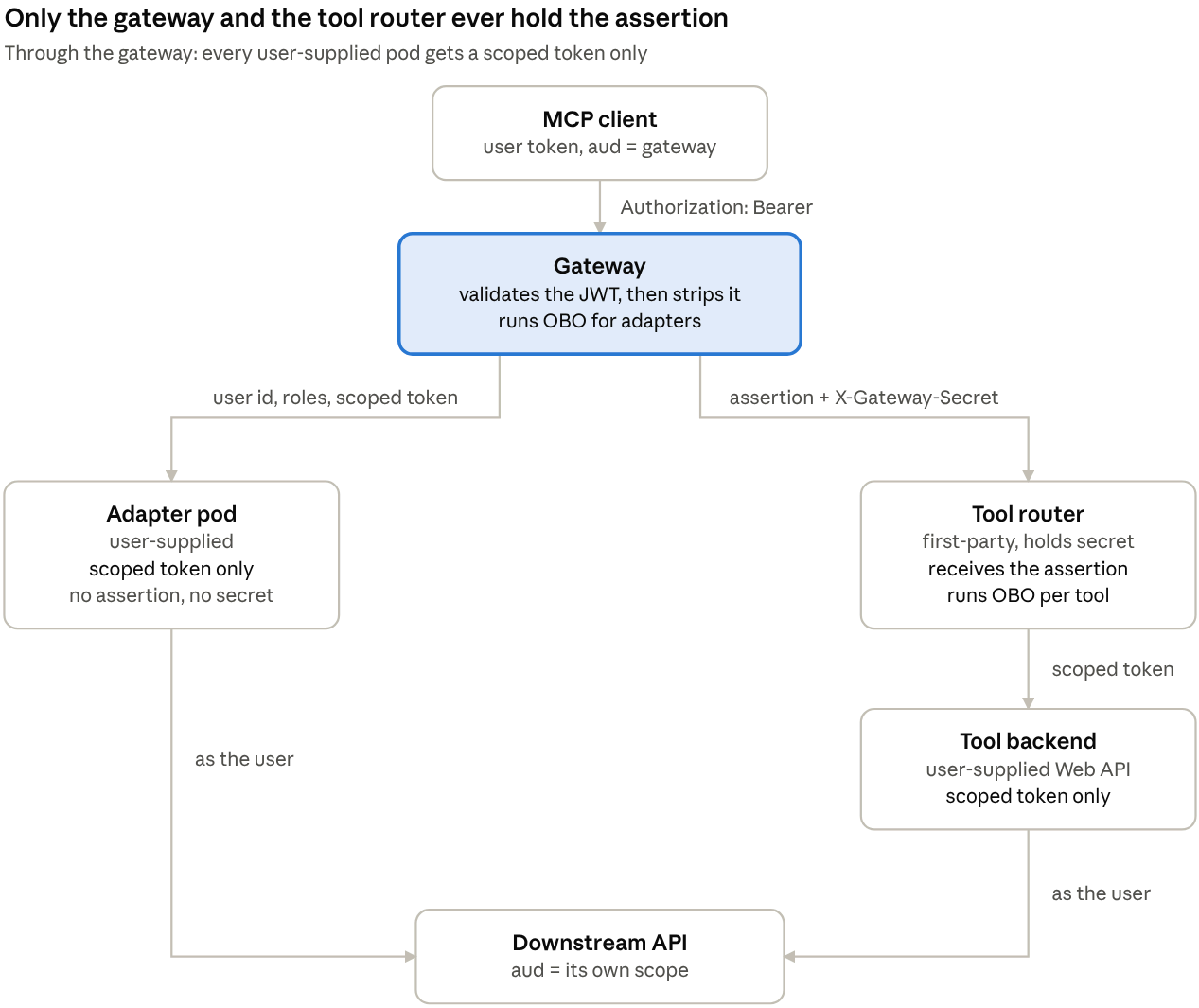
Identity dies before it reaches any backend that could use it. The gateway validates the caller's token, converts the principal into two headers, and discards the token.

Hop	Carries	Source
Client to gateway	`Authorization: Bearer`, audience = gateway	`Service/src/Program.cs:100-117`, `AddMicrosoftIdentityWebApi`
Gateway to adapter pod	`X-Mcp-UserId`, `X-Mcp-Roles`, `Forwarded`	`HttpProxy.cs:72-84`; inbound `Authorization` dropped at `:52-54`
Gateway to tool router	the same, plus `X-Gateway-Secret`	`AdapterReverseProxyController.cs:55`, `forwardGatewaySecret: name is null`
Tool router to tool backend	arguments JSON only, no headers	`HttpToolExecutor.cs:131-139`

Four facts drive the design:

1. No OBO exists anywhere. There is no `ITokenAcquisition`, no `AcquireToken*` call, and the gateway is configured as a pure resource server: `AzureAd` holds `Instance`, `TenantId`, `ClientId` and `Audience`, but no secret or certificate.
2. The gateway is the only component that ever holds the caller's assertion, because the header is stripped at the first proxy hop.
3. The tool router is first-party and already authenticates the gateway hop by shared secret, rejecting anything without it. Adapter pods are user-supplied and deliberately never receive that secret.
4. `ForwardedIdentityHeaders.UserName` is stripped from inbound requests but never injected outbound, so it is effectively dead. Worth deleting or wiring up while in this code.

Two trust paths



token flow · two trust paths, one downstream

The asymmetry is deliberate and follows the trust boundary the code already draws. An adapter pod is user-supplied, so the gateway exchanges the assertion itself and hands the pod nothing but a token scoped to that adapter's declared downstream. The tool router is first-party and already authenticates the gateway hop by shared secret, so it can be given the assertion and mint per-tool tokens itself, which keeps the gateway from having to know which tool a request names.

Why the router, and not the gateway, exchanges for tools: the tool name lives in the MCP request body, which the gateway does not parse. The router already reads it to look up the tool and check `requiredRoles`, so the per-tool scope lookup costs nothing there.

Adapters also run standalone

An adapter image must work unchanged outside the gateway, served directly to a client app. So it needs one code path that resolves two things from whichever mode it finds itself in: who is calling, and what token to send onward.

Concern	Standalone	Behind the gateway
Caller identity	Validate the inbound `Authorization` bearer against its own audience	Read the gateway's signed identity token; the gateway has already enforced `requiredRoles`
Downstream token	Exchange the caller's token itself, using its own app registration	Use the scoped token the gateway supplies
Entra registration	Its own confidential client	None needed

This settles the header question, and settles it against the earlier recommendation. `Authorization` has to keep its standalone meaning, the caller's inbound token that the server validates, so the outbound scoped token must arrive in a distinct header. Reusing `Authorization` would make one header mean "authenticate this caller" in one deployment and "use this for an outbound call" in the other, and a server that got the mode wrong would validate a token minted for a different audience.

Mode detection must be positive and verifiable, never a configuration flag alone. A standalone server is directly reachable, so a server that trusts `X-Mcp-UserId` has no authentication at all: any caller can claim any identity and any role.

The fix is a short-lived signed identity token, minted by the gateway: a JWT with the gateway as issuer, the adapter as audience, the caller's `oid` as subject and the roles as a claim, validated by the adapter against the gateway's JWKS. Present and valid means behind the gateway. Absent means standalone, so fall back to validating `Authorization` directly.

That also closes the spoofing gap in the current design. Adapters cannot verify `X-Mcp-UserId` today because they are deliberately denied the shared secret, and a signed token needs no shared secret, so it gives them a verifiable hop without widening what a compromised pod holds.

Two contract additions follow: a signing key for the gateway with a JWKS endpoint, which its federated credential can back, and a small adapter-side helper package so server authors are not each hand-rolling this validation.

Tool backends deliberately do not get this. They stay callable only through the gateway, so they keep no standalone mode and no caller-facing authentication, and network isolation remains their boundary exactly as it is today. The only thing that changes for them is the scoped downstream token the router attaches before it posts the arguments.

Contract and configuration changes

One new field on the shared resource base carries the whole feature. `ManagedResourceData` already holds `UseWorkloadIdentity`, so `DownstreamScopes` sits beside it and both `AdapterData` and `ToolData` inherit it.

Change	Where	Rule
`DownstreamScopes` (`IReadOnlyList<string>`, default empty)	`ManagedResourceData`, surfaced on `AdapterResource` and `ToolResource`	Empty means the feature is off for that resource, so existing resources are unaffected
Registration gate	`AdapterManagementService`, `ToolManagementService`	Reuse the `IWorkloadIdentityAuthorizer` pattern: only callers in the configured admin roles may set a non-empty value
Immutability	same two services	Changing it after create is rejected, matching the existing `UseWorkloadIdentity` message
Scope allow-list	new `DownstreamTokenSettings`	Gateway operator pins the permitted audiences or scope prefixes; a registration naming anything else is refused
Header to backends	`ForwardedIdentityHeaders`	New constant, see the open decision below

New configuration keys, all under the existing sections:

- `AzureAd:ClientCredentials` — the gateway's confidential-client credential. A federated credential on the existing workload identity avoids adding a secret to the cluster.

- `DownstreamTokenSettings:Enabled` — master switch, default `false` so the gateway's behaviour does not change on upgrade.
- `DownstreamTokenSettings:AllowedScopes` — the operator allow-list.
- `DownstreamTokenSettings:CacheSkewSeconds` — how early to refresh before expiry, default 300.

The management API surface does not change shape. `DownstreamScopes` rides along in the existing `/adapters` and `/tools` create payloads, so the portal and CLI need only a new field.

Implementation steps

Eleven edits in dependency order. Steps 1 to 4 are shared, 5 to 7 deliver adapters, 8 to 10 deliver tools, 11 is documentation.

1. **Contract.** Add `DownstreamScopes` to `ManagedResourceData` and expose it on `AdapterResource` and `ToolResource`, including the `From` / `To` mapping used by the Cosmos and Redis stores. Existing stored documents deserialize to an empty list.
2. **Settings.** Add `DownstreamTokenSettings` next to `WorkloadIdentitySettings` in `Management/src/Authorization`, bound in both `Program.cs` files.
3. **Token service.** New `IDownstreamTokenService` in `Management/src/Authorization` with one method: `TryGetTokenAsync(ClaimsPrincipal caller, string assertion, IReadOnlyList<string> scopes, CancellationToken)`. Implement over `ITokenAcquisition` from `Microsoft.Identity.Web`, backed by `IMemoryCache` keyed on caller object id plus resource name plus the sorted scope list, refreshed `CacheSkewSeconds` before expiry. Return a failure result rather than throwing, so a downstream outage degrades to a clear tool error.
4. **Registration gating.** In `AdapterManagementService` and `ToolManagementService`, validate `DownstreamScopes` against the allow-list, require the admin role for a non-empty value, and reject changes on update. Mirror the existing `UseWorkloadIdentity` checks at `AdapterManagementService.cs:108` and `ToolManagementService.cs:141`.
5. **Gateway credential.** Chain `.EnableTokenAcquisitionToCallDownstreamApi()` onto the `AddMicrosoftIdentityWebApi(azureAdConfig)` call in `Service/src/Program.cs:117`, with `.AddInMemoryTokenCaches()`. This is what turns the gateway from a resource server into a confidential client.

6. **Capture the assertion.** `HttpProxy.CreateProxiedHttpRequest` drops `Authorization` before the principal is read, so the raw bearer value has to be captured before that point and made available to the proxy call site. Do this without changing the method's signature: stash the value on `HttpContext.Items` in middleware and attach the exchanged token in a `DelegatingHandler`, for the reason given under Upstream divergence.
7. **Adapter path.** In `AdapterReverseProxyController.ForwardStreamableHttpRequest`, after the existing `EnsureAdapterReadAccessAsync` check, if the adapter has `DownstreamScopes`, exchange the assertion and attach the result to the outbound request. Named adapters get the scoped token only, never the assertion and never the gateway secret.
8. **Tool path, gateway side.** For the unnamed `/mcp` route, forward the raw assertion to the tool router in a new header alongside the existing `X-Gateway-Secret`. The router is first-party and secret-authenticated, so this does not widen the blast radius the way sending it to an adapter pod would.
9. **Tool path, router side.** In `Tools/src/Program.cs`, read that header and stash the assertion on the request context. Register `IDownstreamTokenService` and a confidential-client credential in this service too.
10. **Tool execution.** In `HttpToolExecutor.ExecuteToolAsync`, after the `CheckAccessAsync` call at `:78`, exchange the assertion for the tool's `DownstreamScopes` and attach the token to the outbound request. Attach it in a `DelegatingHandler` on the executor's `HttpClient` rather than rewriting the `client.PostAsync` call at `:131`, again for the reason under Upstream divergence. Keep the error path: a failed exchange returns a `CallToolResult` with `IsError` set, not an exception.
11. **Documentation.** Update `docs/entra-app-roles.md` with the new field and its admin gate, and `deployment/infra/README.md` with the federated credential and the two trust paths.

While in `HttpToolExecutor`, fix the adjacent bug this work exposes: the non-success branch at `:140-156` returns only `$"Error: Inference server returned {response.StatusCode}"` and logs the body separately. Propagating the backend's response body, or at least a `detail` field, lets a model correct a bad argument instead of retrying blind. A real example: calling the quote tool with `category: "inspirational"` returns a 400 whose body names the three valid values, and the caller sees none of it.

The standalone requirement adds two steps, both in the shared phase. Between steps 5 and 6, give the gateway a signing key and a JWKS endpoint, and mint the signed identity token alongside the existing `X-Mcp-UserId` and `X-Mcp-Roles` headers, which stay for backward compatibility. After step 11, publish a small adapter-side package that validates that token, falls back to validating `Authorization` when it is absent, and exposes the downstream token from either source behind one interface.

Update `sample-servers/mcp-example` to use that package, so the reference implementation demonstrates both deployments rather than only the gateway one.

Entra ID setup

The gateway's existing app registration becomes a confidential client. Nothing about the client-facing side changes, so MCP clients keep using `api://{clientId}/.default` as advertised in the protected-resource metadata.

1. On the gateway app registration, add a federated identity credential pointing at the AKS workload identity the gateway already uses for Azure data access. This avoids a client secret in the cluster, and the infrastructure template already provisions the service account.
2. Grant the gateway app delegated permissions on each downstream API it will exchange for, then grant admin consent. OBO fails with `AADSTS65001` until consent exists, which is the most common setup error here.
3. On each downstream API's app registration, expose the scopes the backends will request, and add the gateway's client id as a known client application so consent flows cleanly.
4. Populate `DownstreamTokenSettings:AllowedScopes` with exactly those scope strings. The allow-list is the operator's control over what any registered resource can ever ask for.
5. Confirm the caller's token audience is the gateway, not a downstream API. OBO requires an assertion issued for the exchanging client; a token the client obtained directly for the downstream API cannot be exchanged.

Two tenant-level constraints are worth checking before committing to this design. Guest or cross-tenant callers may not be able to consent to the downstream API, and Conditional Access policies on the downstream resource are evaluated during the exchange, so a policy requiring a compliant device will fail an exchange made from a cluster.

Threat model and guardrails

The design's one security claim: a user-supplied container never holds a token it can replay anywhere except the downstream API its operator approved for it.

Risk	Guardrail
Adapter replays the caller's token against the gateway	The adapter never receives the assertion. It gets a token whose audience is the downstream API, which the gateway rejects.
Adapter or tool reaches an API it was not approved for	Scopes are per resource, admin-gated at registration, immutable after create, and checked against the operator allow-list.
Tool router is compromised and holds assertions	Already the highest-trust component: it holds the gateway secret and performs the per-tool authorization. Scoped tokens mean an attacker still only reaches approved downstreams. The assertion remains the sharp edge, so prefer gateway-side exchange here if the extra hop can be made to work.
Someone spoofs identity by reaching a pod directly	Addressed by the signed identity token, which needs no shared secret. Until it ships, adapter pods cannot verify that `X-Mcp-UserId` came from the gateway, because they get no secret. Network policy is the only control. Do not let this feature become an argument for giving adapters the gateway secret.
Tokens leak into logs or telemetry	Never log the assertion or the exchanged token, not even truncated. Add the new header names to any telemetry redaction list, and check Application Insights request-header collection.

Risk	Guardrail
Cache returns another caller's token	The cache key must include the caller's immutable object id (`oid`), not a display name or email, plus the resource name and the sorted scope list. This is the single highest-risk line of code in the change and deserves a dedicated test.
Downstream outage becomes an opaque failure	The exchange returns a failure result, surfaced as a `CallToolResult` with `IsError` set or a 502 on the adapter path, never an unhandled exception.

One residual gap to accept knowingly: a tool backend cannot verify that a request reached it from the router rather than from something else in the namespace, since it holds no shared secret and receives no signed token. Network isolation stays its only control, which is acceptable precisely because tools are never exposed outside the cluster. If that decision is ever revisited, this is the first thing to close, and it must be closed before any backend is exposed rather than after.

Testing

Most of this is unit-testable with a faked token service. The exchange itself is not, so plan for a tenant.

Unit tests, extending the three existing test projects:

- `HttpProxyTests` already covers header stripping. Add cases asserting the assertion is never on an outbound adapter request, that a scoped token is attached when the adapter declares scopes, and that nothing is attached when it does not.
- Cache-key tests: two callers with different `oid` and identical scopes get different tokens; the same caller with different scope lists gets different tokens; a token within the skew window is refreshed.
- Registration tests: a non-admin setting `DownstreamScopes` is refused, a scope outside the allow-list is refused, an update that changes the value is refused.
- `HttpToolExecutor` tests: a failed exchange yields `IsError`, and the token reaches the outbound request when scopes are declared.

- A regression test that a resource with empty `DownstreamScopes` produces a byte-identical outbound request to today's.

Only verifiable against a live tenant: that the OBO exchange succeeds, that consent is correctly configured, and that the downstream API accepts the resulting token. Budget for `AADSTS65001` and audience-mismatch errors on the first attempt.

Manual verification sequence: register a tool with one downstream scope as an admin, call it through `/mcp` with a real user token, confirm from the downstream API's own logs that the call arrived as the user and not as the workload identity. That last distinction is the whole point of the feature, and it is the one thing a green unit-test run will not tell you.

Upstream divergence

This repository is `microsoft/mcp-gateway`, MIT licensed and actively maintained. The risk is not permission, it is maintenance, and the working copy is not currently set up to absorb any upstream change at all.

Fix that before writing code:

1. Commit the local changes that already exist. There are seven modified files, an added `global.json`, a 90-line `net10-retarget.patch` and three untracked sample servers, all uncommitted and one `git checkout` from gone.
2. `git fetch --unshallow`. The clone is depth 1, so there is no history, no blame and nothing to rebase against.
3. Branch. The checkout sits on `main` tracking `origin/main` directly. Keep `main` a pristine mirror and do this work on a feature branch, rebasing rather than merging.
4. Pin deployments to a known upstream commit digest so churn never surprises a running cluster.

The plan adds roughly three new files, which cannot conflict, and modifies eight. The modified ones are not equally exposed.

File	Risk	Why
`HttpProxy.cs`	High	Small, security-sensitive and densely commented, so upstream is actively reasoning about it
`HttpToolExecutor.cs`	High	The change lands in the middle of a long method

File	Risk	Why
`Service/Program.cs`, `Tools/Program.cs`	Medium	High-churn by nature, but each edit is one or two lines
Management services, contracts, reverse proxy controller	Low to medium	An additive field and validation that mirrors the existing `UseWorkloadIdentity` checks

Both high-risk edits can be made additive, which is why steps 6 and 10 are written the way they are. Attaching the token in a `DelegatingHandler` registered on the named `Constants.HttpClientNames.AdapterProxyClient`, and on the tool executor's client, is a new file plus one registration line in each case. It survives upstream rewriting either method body, where a changed signature or a restructured `PostAsync` call would not.

The larger risk is strategic rather than textual. Token passthrough and resource-server semantics are an active area in the MCP specification, so this is plausibly something Microsoft builds themselves. If upstream ships its own design, this implementation does not merge, it gets replaced, and every adapter written against `X-Mcp-Downstream-Token` has to change. A header exposed to adapter authors is far more expensive to walk back than a merge conflict.

Two things reduce that risk cheaply. Open an issue upstream describing the need before building, which either reveals an existing roadmap or starts the design conversation that makes upstreaming realistic. Then submit the two uncontroversial fixes on their own: the swallowed error body in `HttpToolExecutor` and the `ForwardedIdentityHeaders.UserName` constant that is stripped but never injected. Both are small, obviously correct, shrink the diff you carry, and establish the relationship before the larger proposal arrives.

Open decisions and rollout

Four decisions are still open, and none of them blocks starting work.

1. **Header name for the scoped token.** Settled by the standalone requirement, in favour of a distinct `X-Mcp-Downstream-Token`. `Authorization` has to stay free to carry the caller's own token when the same image runs outside the gateway. The earlier argument for reusing it, that backend authors get working plumbing for free, is outweighed by one header meaning two opposite things depending on deployment.

2. **Whether to attempt gateway-side exchange for tools.** Forwarding the assertion to the router is simpler and the router is already trusted. Exchanging at the gateway means the assertion never leaves the gateway at all, at the cost of threading per-tool scopes into a route that does not currently know which tool is being called. The router only learns the tool name from the request body. Recommendation: ship the router-side exchange, and revisit if the assertion hop proves contentious in review.
3. **Signed identity token for every adapter, or only standalone-capable ones.** Minting it unconditionally is simpler and closes the existing spoofing gap everywhere, at the cost of a signing key the gateway must manage and rotate. Recommendation: unconditional, since an adapter cannot know at build time which way it will be deployed.
4. **Whether tool backends should also receive caller identity headers.** Out of scope here, but the plumbing added in step 9 makes it nearly free afterwards, and the signed identity token is the right vehicle if you do it.

Rollout is gated on `DownstreamTokenSettings:Enabled`, default off. Deploy the code with the flag off and confirm no behavioural change, then enable it in a non-production deployment and register one tool with one scope, then enable in production. Rollback is the flag, not an image revert, as long as no resource has been registered with scopes. Once resources carry `DownstreamScopes`, a revert to an earlier image leaves the field stored but ignored, which is safe but means those tools silently lose their downstream access.